LibreShelf

Library Management System

CS408 Spring 2026 | Project Specification

Tim | February 27, 2026

Overview

LibreShelf is a self-hostable library management system built for small-to-medium organizations that need something more capable than a spreadsheet but do not need, and could not realistically deploy, a full-scale enterprise system like Koha. It is purpose-built for controlled environments such as schools, prisons, military recreation centers, hospital patient libraries, and nonprofit or community book collections.

The system is built on an offline-first architecture. Book metadata is enriched from the Open Library public API when an internet connection is available, then stored completely in a local SQLite database. Once enriched, the system has no runtime dependency on the internet whatsoever. A built-in ZIP export and import pipeline allows a catalog, including cover images, to be transferred via thumb drive to a system that has never had and will never have an internet connection. This is not a simplified feature; it is a deliberate architectural choice that addresses a real and underserved deployment context.

The system provides two distinct interface modes running from the same application binary. A full staff interface handles all data entry, circulation transactions, patron management, and administration. A separate kiosk interface provides a read-only catalog browse experience for patrons on dedicated terminals. All transactions are handled exclusively by staff; patron terminals have no ability to initiate or modify any data. This separation is intentional and appropriate for controlled environments where the distinction between staff and patron access is operationally significant.

LibreShelf is built with Go 1.24 and the Gin web framework, SQLite with WAL mode enabled, Go's native html/template package, and Bootstrap 5. It deploys as a single compiled binary with no runtime dependencies, making installation and operation accessible to non-technical staff on modest hardware.

Target Audience

LibreShelf is designed for organizations that manage book collections in constrained or controlled environments. The primary target markets are:

- School libraries: classroom or building-level collections where students browse independently and teachers or librarians handle checkout transactions. Hardware is often older or locked-down, and internet connectivity may be unreliable or restricted.
- Prison libraries: correctional facility book collections where inmates browse available titles at kiosk terminals and all checkouts are processed by a library officer or staff member at a central desk. Internet access is typically nonexistent on facility networks, making the offline-first architecture a hard requirement rather than a convenience.
- Military base MWR (Morale, Welfare and Recreation) libraries: recreation lending collections on installations where connectivity varies and simple, low-maintenance software is preferred.
- Hospital patient libraries: lending collections managed by volunteers in healthcare settings where simplicity and reliability are critical.
- Community centers, churches, and nonprofit organizations: informal book collections that have outgrown paper ledgers but have no budget or technical staff for enterprise software.

The common thread across all of these audiences is a need for reliable, simple, offline-capable software with a clear separation between staff and patron access. None of these organizations need MARC records, serial management, acquisitions workflows, or multi-branch synchronization. They need to know what books they have, who has what checked out, and whether a specific title is currently available. LibreShelf is built to answer exactly those questions.

Functionality

Catalog Management

The catalog is the core of the system. Staff can add books manually or by ISBN lookup. When an ISBN is entered and an internet connection is available, the system queries the Open Library API, retrieves full metadata including title, author or authors, publisher, publication year, genre, and description, and downloads the cover image to local storage. The book record is immediately fully populated without manual data entry. Books without an ISBN or without Open Library records can be entered manually. The catalog displays cover images, availability status, and summary metadata in a paginated grid. Search is available by title, author, ISBN, and genre. Availability filtering allows staff and patrons to view only books currently on the shelf.

Circulation

Checkout and return are handled on the book detail page by staff. The checkout form accepts a patron selection and a due date. The return action marks the loan as returned. Overdue status is never stored as a field; it is always computed at query time from the due date compared to the current timestamp, so it is always accurate and cannot become stale. The system uses atomic database transactions with SQLite's BEGIN IMMEDIATE locking to prevent race conditions. It is physically impossible for the same copy of a book to be checked out to two patrons simultaneously. Real-time availability updates are delivered to all connected browsers via Server-Sent Events, so a book checked out at one terminal immediately shows as unavailable on every other open browser window without requiring a page refresh.

Patron Management

Staff can add, edit, and delete patron records. Each patron record stores name, contact information, and join date. The patron detail view shows all currently active loans and complete loan history. Patron names are displayed on all active checkouts in the Admin portal but not in the Kiosk interface.

Admin and Data Pipeline

The admin panel provides three major functions. First, the ISBN enrichment tool: staff enters an ISBN, the system fetches metadata from the Open Library API and downloads the cover image, and the book is added to the catalog fully populated. Second, catalog export: the system packages the entire books table, authors, and all cover images into a ZIP archive that can be saved to a thumb drive. Third, catalog import: an offline system accepts a ZIP upload, extracts it, copies cover images to local storage, and upserts all book records into its local database. The upsert operation means importing an updated catalog twice does not produce duplicate records.

Kiosk Mode

A separate route serves a read-only patron-facing interface suitable for kiosk terminals. The kiosk interface provides the full catalog browse and search experience including cover images, metadata, and real-time availability status. It has no forms, no action buttons, and no ability to modify any data. Patrons can see who has a book checked out and when it is due so they know when it will be available. This interface is appropriate for deployment on dedicated terminals in schools, prisons, and similar environments without any access controls or authentication required on the terminal itself.

Tech Stack

Layer	Technology
Backend Language	Go 1.24+
Backend Framework	Gin (github.com/gin-gonic/gin)
Database	SQLite via modernc.org/sqlite with WAL mode enabled
Frontend Templates	Go html/template with layout pattern
Frontend Styling	Bootstrap 5 via CDN
Real-Time Updates	Server-Sent Events via Go net/http standard library
Image Storage	Local filesystem (static/covers/), filename stored in SQLite
Export/Import	Go archive/zip standard library with no external dependencies
External API	Open Library API (openlibrary.org/developers/api)
Testing	Go testing package and net/http/httptest
CI	GitHub Actions on every push to main
Deployment	Single binary, systemd service, and nginx reverse proxy on Ubuntu EC2

SQLite was chosen deliberately for this project rather than PostgreSQL. The offline-first architecture makes SQLite the correct tool: the entire database is a single file that travels in a ZIP archive on a thumb drive. A PostgreSQL installation requires a running server process, system configuration, and a network connection between the application and the database engine, none of which are compatible with offline deployment. SQLite with WAL mode enabled supports concurrent reads without blocking and serializes writes safely, which is entirely appropriate for a single-instance library system with a small staff. If the system were ever extended to a multi-branch networked architecture, the database layer is written against Go's standard database/sql interface and a driver swap to PostgreSQL would not require a full rewrite.

Database Schema

The schema uses four primary tables and one join table. All foreign key constraints are enforced. WAL journal mode is set at connection open time via PRAGMA.

books

Column	Type	Notes
id	INTEGER	PRIMARY KEY AUTOINCREMENT
isbn	TEXT	UNIQUE, may be null for books without ISBN
title	TEXT	NOT NULL
publisher	TEXT	
year	INTEGER	
description	TEXT	
cover_filename	TEXT	Filename only, file lives in static/covers/
genre	TEXT	
quantity_total	INTEGER	DEFAULT 1
quantity_available	INTEGER	DEFAULT 1, decremented on checkout, incremented on return

authors

Column	Type	Notes
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL UNIQUE

book_authors (join table)

Column	Type	Notes
book_id	INTEGER	FOREIGN KEY REFERENCES books(id)
author_id	INTEGER	FOREIGN KEY REFERENCES authors(id)

patrons

Column	Type	Notes
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL
email	TEXT	
phone	TEXT	
joined_date	TEXT	ISO 8601 date string

loans

Column	Type	Notes
id	INTEGER	PRIMARY KEY AUTOINCREMENT
book_id	INTEGER	FOREIGN KEY REFERENCES books(id)
patron_id	INTEGER	FOREIGN KEY REFERENCES patrons(id)
checked_out_at	TEXT	ISO 8601 timestamp
due_date	TEXT	ISO 8601 date string
returned_at	TEXT	NULL if still checked out

Overdue status is never stored. It is always computed at query time: returned_at IS NULL AND due_date < CURRENT_TIMESTAMP. This means overdue status is always accurate and cannot become stale regardless of when the page is loaded.

Pages

Page	Route	Description
Dashboard	/	System stats, active loans count, overdue alerts, recent activity feed
Catalog	/catalog	Paginated book grid with search, genre filter, availability filter, cover thumbnails

Book Detail	/books/:id	Full metadata, availability, checkout/return form, loan history for that title
Patrons	/patrons	Patron list with add form; patron detail shows active loans and full history
Admin	/admin	ISBN lookup, Open Library enrichment, ZIP export, ZIP import, system information
Kiosk	/kiosk	Read-only catalog browse for patron-facing terminals, no forms, no actions

All pages share a common header and footer rendered from the layout template. The header includes navigation to all staff pages. The kiosk route renders a separate stripped-down layout with no navigation to staff functions and no action elements of any kind.

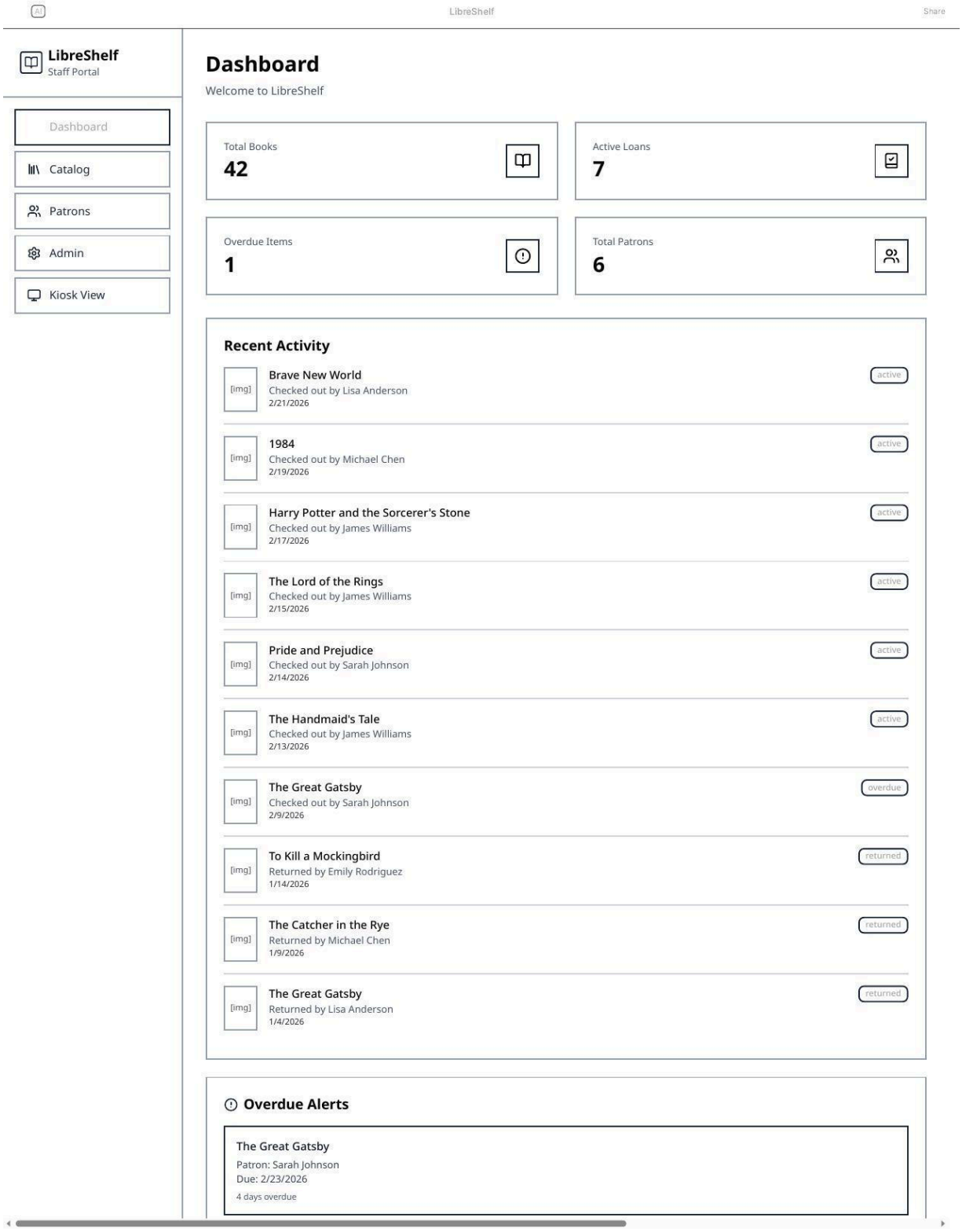
Media

All book cover images are fetched programmatically from the Open Library Covers API during the ISBN enrichment process. Images are downloaded and stored as JPEG files in the static/covers/ directory on the server filesystem, named by ISBN. The database stores only the filename as a short text string. A fallback placeholder image is used for books that do not have a cover available from Open Library or that were entered manually without a cover. During ZIP export, cover images are included in a covers/ subdirectory within the archive alongside the catalog JSON. On import, images are extracted to the receiving system's static/covers/ directory, giving offline installations full visual fidelity identical to the source system.

Wireframes

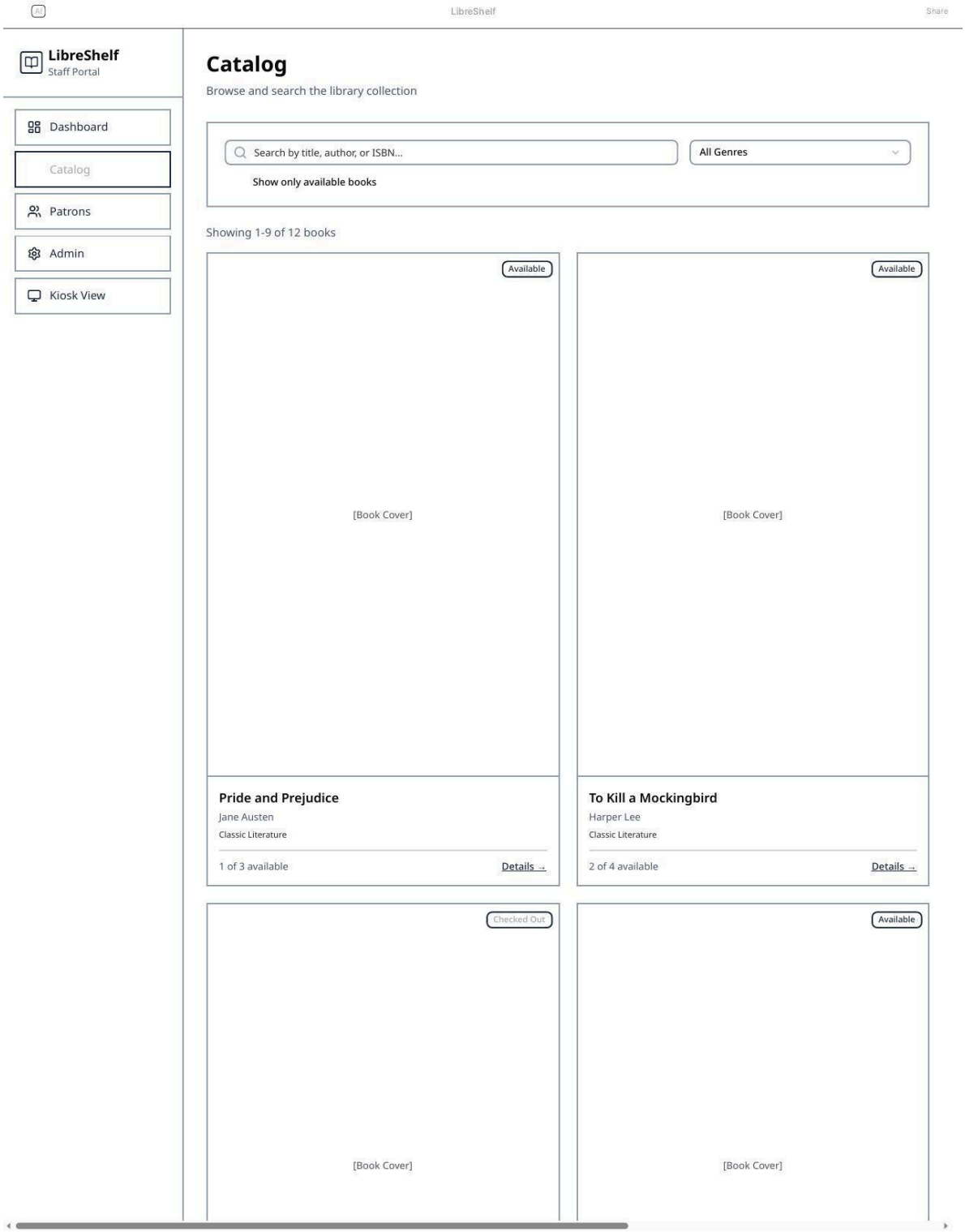
Wireframes were created in Figma. All six pages are represented below. The staff interface pages share a consistent sidebar navigation. The kiosk page intentionally omits all navigation and action elements to reflect its read-only patron-facing purpose.

1. Dashboard /



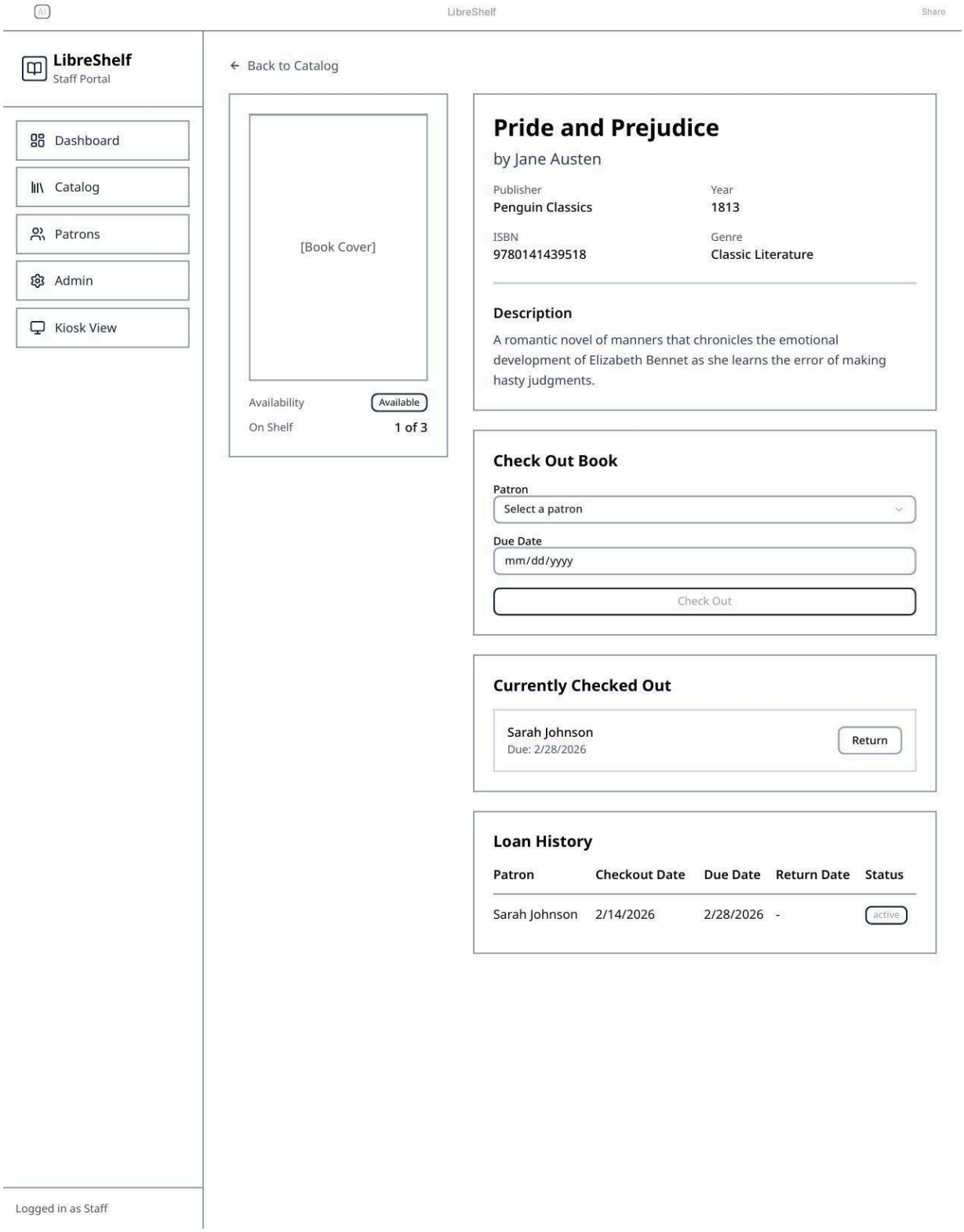
Staff landing page showing system statistics, recent activity feed, and overdue alerts.

2. Catalog */catalog*



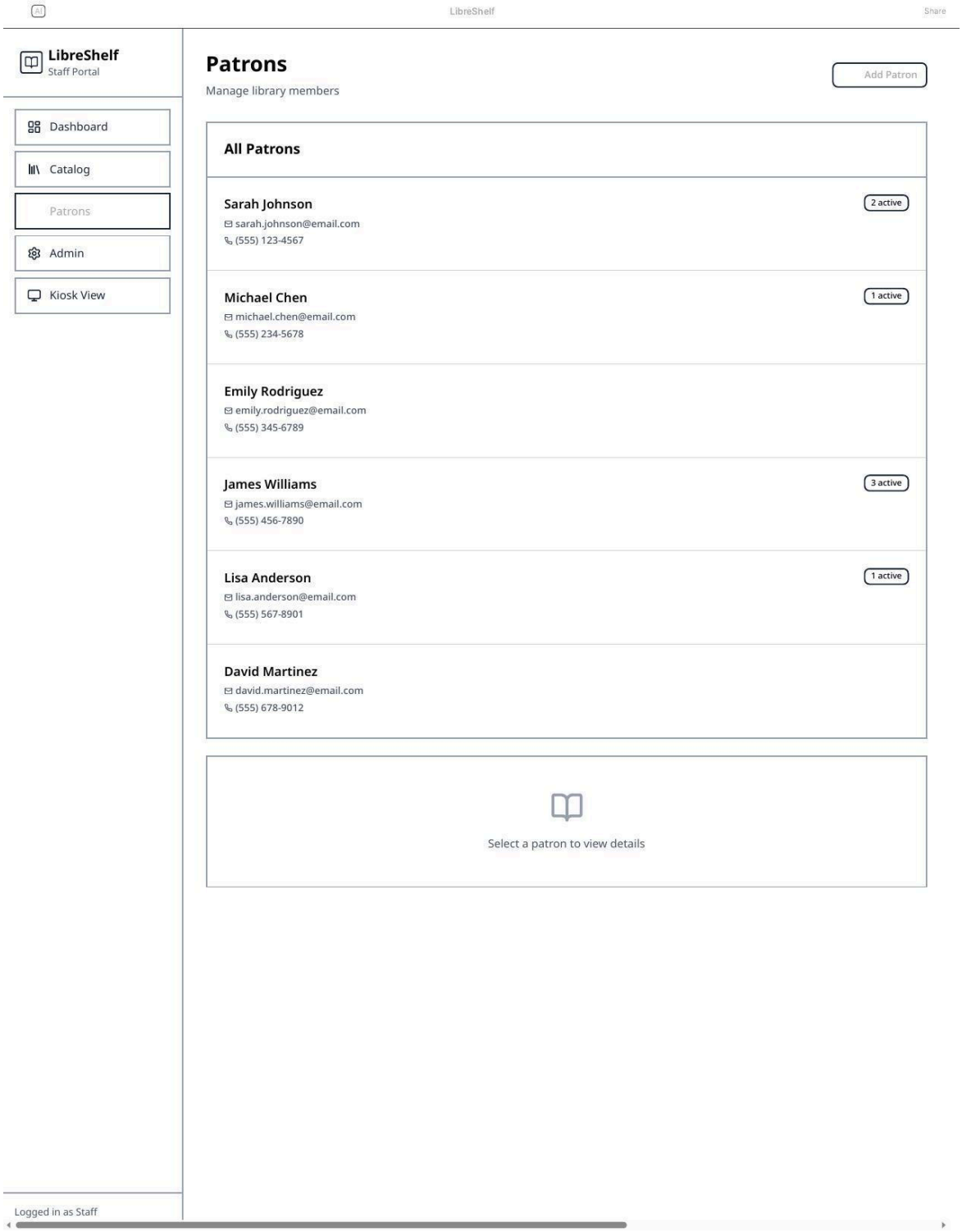
Paginated book grid with search bar, genre filter, and availability toggle. Each card shows cover image and availability badge.

3. Book Detail */books/:id*



Full book metadata alongside checkout form, active loans with return button, and complete loan history table.

4. Patrons */patrons*



Patron list with contact information and active loan counts. Staff can add, view, and manage patron records.

5. Admin */admin*

ISBN lookup tool, ZIP export with file contents listed, ZIP import with drag-and-drop upload, and system statistics.

6. Kiosk */kiosk*

Read-only patron-facing catalog. No sidebar, no action buttons. Shows availability and checkout info but allows no modifications.

Automated Testing

Testing uses Go's built-in testing package and net/http/httptest. No external testing frameworks are required. The CI pipeline is already configured via GitHub Actions and runs all tests automatically on every push to main.

The testing strategy follows a three-layer pyramid. Unit tests cover the database layer in isolation: all CRUD operations, the atomic checkout transaction, overdue computation logic, and the upsert behavior on import. These tests use an in-memory SQLite database initialized fresh for each test function so they never touch real data and run in milliseconds. Integration tests cover the HTTP handler layer: catalog search and filter, checkout and return flows, patron creation and deletion, ZIP export generation, and ZIP import processing. End-to-end tests covering the Server-Sent Events real-time update pathway are included as a stretch goal.

The GO_ENV=test environment variable gates test-only utilities such as in-memory database initialization and seed data insertion, keeping test infrastructure completely separate from production code paths.

EC2 Setup Scripts

scripts/install.sh

Installs all required system packages on a fresh Ubuntu EC2 instance. Runs apt update and installs nginx. Copies the compiled Go binary and the systemd service unit file to the correct system locations. Enables and starts the systemd service. Configures nginx as a reverse proxy forwarding port 80 to the application on port 3000. Removes the nginx default site and enables the LibreShelf site configuration.

scripts/configure.sh

Handles application-level setup after the binary is in place. Creates the data/ directory for the SQLite database file and the static/covers/ directory for book cover images with correct ownership for the service user. Writes the environment configuration file with PORT, DATA_DIR, and DB_NAME values. Runs the database migration command which creates all tables if they do not exist. This script is idempotent, meaning running it multiple times on an existing installation does not overwrite data or reset configuration.

Schedule

Checkpoint	Week	Deliverables
CP1	Weeks 1-2	Project structure complete. All 6 routes registered and returned rendered pages. Bootstrap layout with shared header and footer. Placeholder data on all pages. Database schema implemented with migrations running on startup. GitHub Actions CI passing.

CP2	Week 3	Full database layer implemented and tested. All CRUD operations for books, authors, patrons, and loans. Atomic checkout transaction with BEGIN IMMEDIATE locking verified by concurrent test. Overdue computation query tested. WAL mode confirmed active. Unit test coverage for all db.go methods.
CP3	Week 4	Catalog page fully functional. Paginated book grid with cover image display. Search by title, author, and ISBN working. Genre and availability filters working. Book detail page showing full metadata and loan history. All catalog HTTP handler integration tests passing.
CP4	Week 5	Circulation system complete. Checkout and return forms are functional with atomic transaction protection. Patron management page with add, edit, delete, and loan history view. Dashboard showing live stats. Overdue highlighting visible on dashboard and patron detail.
CP5	Week 6	Admin panel complete. Open Library API ISBN lookup fetching metadata and downloading cover image. ZIP export generating correctly structured archive with catalog JSON and covers directory. ZIP imports uploading, extracting, and upserting records on a test offline instance. End-to-end export and import round trip verified.
CP6	Week 7	Server-Sent Events implemented. Availability badge updates pushed to all connected browsers on checkout and return events. Kiosk interface complete at /kiosk with full catalog browse and real-time availability but no staff functions. SSE behavior tested across two simultaneous browser sessions.
CP7	Week 8	EC2 deployment is live and publicly accessible. Both install and configure scripts tested on a fresh instance. All tests passing in CI. Final documentation complete. Application performance verified on EC2 free tier hardware.